

Table of Contents

Week 6 — Inheritance

The Second Pillar: Building on What Already Exists

Lesson Overview

Hour Plan

Recap of Week 5 (5 min)

Part 1 — What Is Inheritance? (13 min)

Open with a question (3 min)

Definition

The Core Purpose — Four Reasons Inheritance Exists

Real-World Hierarchy Examples

The No-Inheritance Problem — A Concrete Story

Part 2 — The IS-A Test: When to Use Inheritance (10 min)

Open with a question (2 min)

The IS-A Test

IS-A vs HAS-A — The Most Common Mistake

Depth — How Many Levels?

Part 3 — How Inheritance Works: Parent First (12 min)

Open with a question (2 min)

The Parent-First Rule

What the Child Passes Up to the Parent

Two Levels of Access: Shared vs Truly Private

Part 4 — What a Child Adds, What It Inherits (10 min)

The Inheritance Equation

What a Child Receives Automatically

The instanceof Idea — A Child IS Also Its Parent

Single Inheritance — One Parent Only

Part 5 — Java as Illustration (8 min)

Reading an Inheritance Hierarchy as Concepts

Reading as Plain English

Demonstrating What Is Inherited for Free

Test Checkpoint

Common Misconceptions

- Basic — IS-A or HAS-A?
- Intermediate — Design the Hierarchy
- Advanced — When IS-A Goes Wrong

Lesson Summary

Homework

Week 6 — Inheritance

The Second Pillar: Building on What Already Exists

Date: Wednesday, 8 July 2026 | **Time:** 6:30 PM – 8:30 PM | **Course:** Object-Oriented Programming Concepts

Lesson Overview

Item	Detail
Topic	Inheritance — how classes build on each other through IS-A relationships
Date & Time	Wednesday, 8 July 2026 · 6:30 PM – 8:30 PM
Duration	2 hour
Format	Online (Google Meet) — Concept explanation + real-world discussion + light illustration
Prerequisites	Week 4 (Encapsulation), Week 5 (Constructors)
Focus	OOP concept understanding — Java used only as illustration
Outcome	Students can explain inheritance and why it exists, correctly apply the IS-A test, describe what a child class inherits from its parent, explain the parent-first setup rule, and distinguish what is shared vs what is unique to each class

Hour Plan

00:00 – 00:05	Recap of Week 5 + Homework check
00:05 – 00:18	Part 1 – What Is Inheritance?
00:18 – 00:28	Part 2 – The IS-A Test: When to Use Inheritance
00:28 – 00:40	Part 3 – How Inheritance Works: Parent First
00:40 – 00:50	Part 4 – What a Child Adds, What It Inherits
00:50 – 00:58	Part 5 – Java as Illustration (light)
00:58 – 01:00	Checkpoint + Homework

 **Google Meet tip:** Keep `Week6_Summary_Examples.md` open in a second tab — it has a completely different set of real-world stories (job roles and vehicle types) ready for every part, in case a student needs a second angle.

Recap of Week 5 (5 min)

Ask students verbally:

1. "What problem does a constructor solve? What was wrong with the old multi-step setup approach?"
→ *Incomplete objects could exist. Constructor ensures the object is complete and valid before being used.*
2. "Why is it better for all setup paths to hand off to a single master constructor?"
→ *One place to fix validation, one place to update — no risk of missing a path.*
3. "From homework — you designed three Smartphone setup paths. Which one was the master? What defaults did the other paths fill in?"
→ *The full path (brand + model + storageGB). Others filled in missing values with defaults.*
4. Transition question: "If Sokha is a Student, is she also a Person? What else is she a type of?"
→ *Yes — a Student IS-A Person. This is today's entire lesson in one question.*

Part 1 — What Is Inheritance? (13 min)

Open with a question (3 min)

*"Think about your own family. What did you inherit from your parents or grandparents?
Maybe their eye color. Maybe a talent for music. Maybe the same laugh.*

You were born with those traits already — you didn't have to learn them from scratch.

You also have things that are uniquely yours — traits your parents don't have.

In OOP, classes work the same way."

Definition

Inheritance is the OOP mechanism where a **child class** automatically receives all the shared attributes and actions from a **parent class** — without those things needing to be written again.

The child starts with everything the parent already has, and can then add its own unique attributes and actions on top.

The Core Purpose — Four Reasons Inheritance Exists

REASON 1 – AVOID REPETITION

Define shared information and actions **ONCE** in the parent.
All children automatically have them – no copy-pasting.

Without inheritance: Student has name, age, introduce().
Teacher also has name, age, introduce().
The same things written **TWICE** (or more).

With inheritance: Person has name, age, introduce().
Student and Teacher **INHERIT** all three.
Written **ONCE**. Used everywhere.

REASON 2 – MODEL THE REAL WORLD NATURALLY

The real world has natural categories and subcategories.
A Student **IS-A** Person. A Toyota **IS-A** Car. A Dog **IS-AN** Animal.
OOP lets code reflect these natural hierarchies.

REASON 3 – EXTEND EASILY

To add a new type to the system (e.g., Administrator),
just create a new child class. No existing code needs changing.
The new type automatically has everything the parent had.

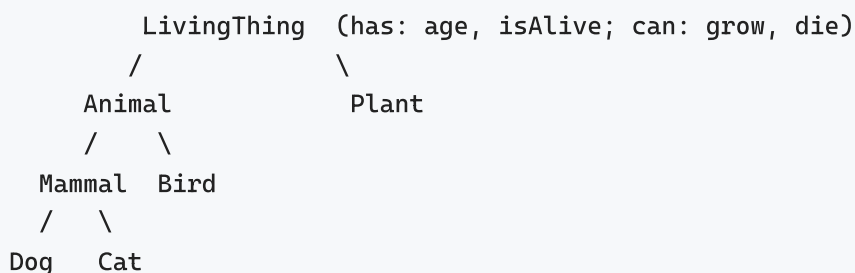
REASON 4 – FOUNDATION FOR POLYMORPHISM

Next month (Week 9), we will see how inheritance allows
very powerful "write once, works for all children" behavior.
It is built on the foundation we lay today.

Real-World Hierarchy Examples

Work through each one. Ask students: "What does the parent have that all children share?"

EXAMPLE 1 – LIVING THINGS



Every Dog **IS-A** Mammal **IS-AN** Animal **IS-A** LivingThing.

A Dog automatically has everything a LivingThing has.

EXAMPLE 2 – UNIVERSITY PEOPLE

```
      Person (has: name, age; can: introduce())
    /      \
  Student  Teacher
(adds:      (adds:
  major,    subject,
  gpa)      yearsExp)
```

Both Student and Teacher are Persons.
Both automatically have name, age, and introduce().

EXAMPLE 3 – VEHICLES

```
      Vehicle (has: brand, speed; can: move(), stop())
    /   |   \
  Car  Truck Motorcycle
```

Every Car IS-A Vehicle.
Every Car automatically has brand, speed, move(), stop().
A Car only needs to add what makes it uniquely a Car.

The No-Inheritance Problem — A Concrete Story

"Imagine a university system with NO inheritance. The programmer writes a Student class with: name, age, email, phoneNumber, introduce(), contactInfo(). Then they write a Teacher class and realize: 'A teacher also has name, age, email, phoneNumber.' They copy-paste everything.

Three months later, a new rule: all people in the system need a new attribute — nationalId. The programmer must now find and update Student, Teacher, AND any other class that represents a person — one by one, hoping they don't miss one."

THE COPY-PASTE PROBLEM:

```
Student:  name, age, email, phone  ← copied
Teacher:  name, age, email, phone  ← copied
Admin:    name, age, email, phone  ← copied
```

New rule: add nationalId to all persons
→ Must update 3 classes separately

WITH INHERITANCE:

Person: name, age, email, phone, nationalId

Student: (inherits everything from Person) + major, gpa

Teacher: (inherits everything from Person) + subject

Admin: (inherits everything from Person) + department

New rule: add nationalId

→ Update Person ONCE

→ ALL children automatically get it

Part 2 — The IS-A Test: When to Use Inheritance (10 min)

Open with a question (2 min)

"Is a Student a Person? Is a Car an Engine? Is a Library a Book?"

Think carefully — the answer to each question determines whether we should use inheritance."

The IS-A Test

Before using inheritance, always ask: "Is a [Child] truly a [Parent]?"

If the answer is genuinely YES — the child IS a more specific version of the parent — use inheritance.

If the answer is NO — use a different relationship (the child might have or use the other thing instead).

THE IS-A TEST – APPLIED

Question	Answer	Decision
Is a Student a Person?	YES 	Student inherits from Person
Is a Teacher a Person?	YES 	Teacher inherits from Person
Is a Dog an Animal?	YES 	Dog inherits from Animal
Is a Car a Vehicle?	YES 	Car inherits from Vehicle
Is a SavingsAccount a BankAccount?	YES 	SavingsAccount inherits from BankAccount
Is a Car an Engine?	NO 	A Car HAS-AN engine (Week 12)
Is a Library a Book?	NO 	A Library HAS many Books
Is a Student a Department?	NO 	A Student BELONGS TO a Department
Is a Phone a Battery?	NO 	A Phone HAS-A Battery

IS-A vs HAS-A — The Most Common Mistake

This is the single most frequent beginner inheritance error. Use a real story.

"A programmer is building a car system. They think: 'A car has an engine, and I want a Car object to be able to call engine.start(). The quickest way is to make Car extend Engine — then Car automatically has all of Engine's methods.'

But wait — IS a Car an Engine? No! A car is NOT a type of engine. A car USE an engine, or CONTAINS an engine. Using inheritance here would be like saying 'a house IS a window' just because houses HAVE windows. It is simply not true."

IS-A vs HAS-A – THE RULE

IS-A = the child is a MORE SPECIFIC VERSION of the parent

- Use inheritance
- "A Dog IS a more specific type of Animal"

HAS-A = the object CONTAINS or USES another object

- Use a field reference (Week 12: Object Relationships)
- "A Car HAS an engine inside it"

TEST YOURSELF:

- "Is a Sandwich a Kitchen?" → No → HAS-A (kitchen makes sandwiches)
 - "Is a GraduateStudent a Student?" → Yes → IS-A (inheritance)
 - "Is a Teacher a Classroom?" → No → HAS-A (teacher works in a classroom)
 - "Is a SavingsAccount a BankAccount?" → Yes → IS-A (inheritance)
-

Depth — How Many Levels?

RECOMMENDED – 2 to 3 levels deep

Person → Student → GraduateStudent
Clear, understandable, manageable

BE CAREFUL – more than 4 levels

LivingThing → Animal → Mammal → Primate → Human → Employee
→ Every change to LivingThing ripples down all 5 levels
→ Hard to understand which class provides which behavior
→ Adding a new level later becomes very complex

Part 3 — How Inheritance Works: Parent First (12 min)

Open with a question (2 min)

"When a new employee joins a company, on their first day they go through company-wide orientation: ID card, security access, payroll registration — things every single employee needs regardless of their department.

THEN they go through their specific department training: how a software engineer works here, how a finance officer works here. This specific part is different for everyone.

Notice the order: company-wide setup first, specific setup second. Always."

The Parent-First Rule

When creating a child object, the parent part of the object is always set up FIRST.

The child's own specific setup runs second.

This order is not optional — the child cannot set itself up before the parent is ready.

WHY PARENT MUST COME FIRST:

A Student is a Person. The "Person part" of a Student includes name, age, and email. These exist INSIDE every Student object – they are the foundation.

If the Person part is not set up first, the Student part has no foundation to build on. It would be like building the second floor of a house before the first floor exists.

ORDER:

1. Parent constructor runs → sets name, age, email
 2. Child constructor runs → sets major, gpa, studentId
(it can now also ACCESS name and age – they already exist)
-

What the Child Passes Up to the Parent

"When the hotel chain headquarters opens a new branch, the branch manager still must register the branch with headquarters — providing the standard company-wide information. The branch cannot exist without being properly registered at the top level.

In OOP, when a child is created, it must provide the parent with everything the parent needs to set itself up."

PARENT-CHILD SETUP FLOW:

Creating a Student named "Sokha", age 20, major "CS":

Step 1: Student's setup begins

Step 2: Student immediately says: "Parent – here is your part:
name='Sokha', age=20. Please set yourself up first."

Step 3: Person's setup runs with name="Sokha", age=20

Step 4: Person is fully set up

Step 5: Student continues with its OWN setup: major="CS"

Step 6: Student is complete – the full object is ready

Two Levels of Access: Shared vs Truly Private

Last week (encapsulation), we made everything private. But inheritance introduces a new question: should a child class be able to access its parent's data directly?

TRULY PRIVATE (sealed – child cannot touch directly):

The parent keeps some information strictly to itself.

Even its own children must use the parent's getters to access it.

Example: a parent's salary – even family members ask, they don't just open the pay slip directly.

FAMILY-ACCESSIBLE (shared with children only, not strangers):

The parent shares some information directly with its children – but not with completely unrelated code.

Example: family recipes – shared within the family, not published to the whole world.

IN OOP:

Truly private → only the class itself can touch it

Family-access → this class AND its children can touch it

Public → everyone can touch it

Part 4 — What a Child Adds, What It Inherits (10 min)

The Inheritance Equation

CHILD CLASS = EVERYTHING FROM PARENT + CHILD'S OWN ADDITIONS

Person has:	name, age	→ introduces(), contactInfo()
	↓	(inherited by both children below)
Student has:	name, age + major, gpa	→ introduce(), study(), enroll()
	↑	
	SHARED	UNIQUE TO STUDENT
Teacher has:	name, age + subject, experience	→ introduce(), teach()
	↑	
	SHARED	UNIQUE TO TEACHER

What a Child Receives Automatically

A CHILD CLASS AUTOMATICALLY GETS:

- ✓ All public attributes from the parent (and from all ancestors)
- ✓ All public actions from the parent (can use without rewriting)
- ✓ All family-accessible (protected) attributes from the parent
- ✓ Everything from the parent's parent, grandparent, etc.

A CHILD CLASS DOES NOT AUTOMATICALLY GET:

- ✗ Truly private attributes from the parent
(they EXIST inside the object – but the child must ask via the parent's public "read" action to access them)
- ✗ The parent's setup action (constructor) – the child must explicitly trigger it during its own setup

The instanceof Idea — A Child IS Also Its Parent

"A Student IS-A Person. That means a Student object is BOTH a Student AND a Person at the same time. It carries everything a Person has, plus its own Student-specific things."

STUDENT OBJECT – TWO TRUTHS AT ONCE:

```
s1 = Student("Sokha", 20, "CS")
```

IS s1 a Student? YES – it has major, gpa, study(), enroll()

IS s1 a Person? YES – it also has name, age, introduce()

A student is BOTH simultaneously.

This is the IS-A relationship in action.

Practical use:

"Is this thing a Person?" → YES (all Students are Persons)

"Is this thing a Student?" → YES (specifically a Student)

"Is this thing a Teacher?" → NO (it's a Student, not a Teacher)

Single Inheritance — One Parent Only

IN OOP (AND JAVA), A CLASS CAN ONLY HAVE ONE PARENT:

Allowed:

Student inherits from Person 

GraduateStudent inherits from Student 

(which also inherits Person – a chain of 3 levels)

NOT allowed:

Student inherits from BOTH Person AND Animal



WHY?

If Person has an introduce() and Animal also has an introduce() with different behavior – and Student inherits both – which version does Student get?

No clear answer → the language prevents this confusion.

SOLUTION (Week 11 – Interfaces):

A class can only have ONE parent class.

But it can promise to support MANY separate capability contracts. This is the interface solution.

Part 5 — Java as Illustration (8 min)

Reminder: Java is just how we write these concepts down. Focus on the concept, not the syntax.

Reading an Inheritance Hierarchy as Concepts

```
// PARENT class - defines what every Person has and can do
public class Person {
    protected String name;    // "family-accessible" - children can touch this directly
    protected int    age;

    // Parent's setup action - triggered first by all children
    public Person(String name, int age) {
        this.name = name;
        this.age  = age;
    }

    // Action every Person can do - shared by all children for free
    public void introduce() {
        System.out.println("Hi, I am " + name + ", age " + age);
    }
}

// CHILD class - inherits everything from Person, adds its own things
public class Student extends Person {    // "extends" = IS-A relationship
    private String major;

    public Student(String name, int age, String major) {
        super(name, age);    // "trigger the parent's setup first"
        this.major = major; // then add my own unique setup
    }

    // Student's OWN unique action - not in Person
    public void study() {
        System.out.println(name + " is studying " + major);
        // name is accessible because it is "protected" in Person
    }
}

// ANOTHER CHILD - also inherits from Person
public class Teacher extends Person {
    private String subject;

    public Teacher(String name, int age, String subject) {
        super(name, age);    // same parent-first rule
        this.subject = subject;
    }
}
```

```

    public void teach() {
        System.out.println(name + " teaches " + subject);
    }
}

```

Reading as Plain English

CONCEPT TRANSLATION TABLE

Java code	OOP concept meaning
extends Person	"Student IS-A Person – inherits from it"
super(name, age)	"Trigger the parent's setup first"
protected name	"Family-accessible – children can use directly"
student.introduce()	"Works – inherited from Person, no rewrite"
student.study()	"Works – uniquely Student's own action"
student instanceof Person	"True – Student IS-A Person"

Demonstrating What Is Inherited for Free

```

Student sokha = new Student("Sokha", 20, "CS");

sokha.introduce(); // → "Hi, I am Sokha, age 20"
                  // ↑ This action is from Person
                  // Student never wrote it – it was inherited

sokha.study();    // → "Sokha is studying CS"
                  // ↑ This action is uniquely Student's

```

Test Checkpoint

- ☐ Can explain what inheritance is in one sentence using a real-world analogy (not code)
- ☐ Can apply the IS-A test to 5 new examples and correctly decide whether inheritance applies
- ☐ Can explain why the parent part must be set up before the child part (using the employee orientation story or similar)
- ☐ Can explain the difference between "truly private" and "family-accessible" data
- ☐ Can explain why a Student is both a Student AND a Person simultaneously

Common Misconceptions

Misconception	Clarification
"Inheritance means copying code"	No — inheritance means the child automatically <i>has</i> the parent's things without them being written again. There is no copying.
"If I use inheritance, the parent data is duplicated inside the child"	No — the parent's attributes exist once inside the child object. There is one object, with a parent section and a child section.
"I can use inheritance whenever I want a class to have another class's methods"	Only use inheritance for a genuine IS-A relationship. If you just want to <i>use</i> another class's features, use a field reference (HAS-A).
"The child can access all of the parent's attributes directly"	Only family-accessible (protected) and public ones. Truly private attributes exist in the object but the child must ask via the parent's getter — it cannot touch them directly.
"Inheritance means the parent class changes when the child does"	Never. Adding a new child class does not change the parent at all. A new employee joining the company does not change company headquarters.

Extension Challenges

Basic — IS-A or HAS-A?

For each pair, decide: should this be **IS-A (inheritance)** or **HAS-A (field reference)**? Explain your reasoning in one sentence.

1. GraduateStudent and Student
2. Car and Engine
3. SavingsAccount and BankAccount
4. Smartphone and Battery
5. Doctor and Person
6. Hospital and Doctor

🟡 Intermediate — Design the Hierarchy

Design the following class hierarchy in plain English — no code needed:

Starting class: `Vehicle` with attributes: `brand`, `speed`, `fuelType` and actions: `move()`, `stop()`

Add three children:

- `Car` — adds `numberOfDoors`, `hasTrunk`
- `Motorcycle` — adds `hasSidecar`
- `Truck` — adds `cargoCapacityKg`

For each child:

- List what it inherits from `Vehicle` (without rewriting)
- List what is uniquely its own
- Apply the IS-A test to confirm the relationship is valid

🔴 Advanced — When IS-A Goes Wrong

Consider this scenario:

A programmer is building a shape-drawing system. A `Square` is a special kind of `Rectangle` where all four sides are equal. The programmer decides: `Square extends Rectangle`. This passes the IS-A test — a `Square` IS-A `Rectangle`.

However, `Rectangle` has a `setWidth(double w)` and `setHeight(double h)` action. For a `Rectangle`, these work independently. But for a `Square`, changing the width must ALSO change the height (they must stay equal).

Write a short analysis:

- Why does `Square extends Rectangle` seem correct?
- What goes wrong when a `Square` inherits `setWidth()` and `setHeight()` from `Rectangle`?
- What does this tell us about the IS-A test: is it always enough on its own?
- What might be a better design?

Lesson Summary

TODAY'S KEY TAKEAWAYS

1. INHERITANCE lets a child class automatically receive all the shared attributes and actions from a parent class – with no repetition.
2. Use the IS-A TEST before applying inheritance.

If NO → use a field reference (HAS-A) instead.

3. PARENT FIRST: when a child object is created, the parent part is always set up before the child's own part.
Like company orientation before department training.
 4. A child inherits all PUBLIC and FAMILY-ACCESSIBLE (protected) things from the parent.
TRULY PRIVATE things exist in the object but the child must access them through the parent's public getter.
 5. A child IS ALSO its parent simultaneously.
A Student is both a Student AND a Person.
This is the IS-A relationship made real.
 6. ONE PARENT ONLY – a class can inherit from only one other class. Multiple inheritance is not allowed.
(Multiple capability contracts via interfaces – Week 11.)
 7. Inheritance is the foundation for Polymorphism (Week 9):
one action name, many different behaviors per child type.
-

Homework

Design a three-level class hierarchy for the university system — plain English only, no code required:

1. **Parent:** `Person` — attributes: `name`, `age`, `email` | actions: `introduce()`, `contact()`
2. **Two children of Person:**
 - `Student` — adds: `major`, `studentId`, `gpa` | adds: `study()`, `enrollInCourse()`
 - `Teacher` — adds: `subject`, `yearsExperience` | adds: `teach()`, `gradeExam()`
3. **One grandchild (child of Student):**
 - `GraduateStudent` — adds: `thesisTopic`, `supervisorName` | adds: `presentThesis()`

For each class, write:

- What it inherits (without rewriting)
- What is uniquely its own

Apply the IS-A test at each level to confirm every relationship is valid.

Optional bonus: Write the three Java class declarations (just the class header + constructor + one unique method each), following today's illustration as a guide.

Bring your design to Week 7 — next week we explore what happens when a child wants to *replace* a parent's behavior rather than just inherit it. We'll use your hierarchy to see this in action.

